Components, props, and state

Module 3 - Lesson 1

Overview

React applications are trees of components. Props flow data down from parents, state holds data that changes over time, and together they make the UI a pure function of your data.

Key Concepts

- A component is a function that returns JSX describing the UI.
- Props are read-only inputs passed from a parent; a component must not modify its own props.
- State is data owned by a component that can change; updating it re-renders the component.
- The useState hook returns the current value and an updater; updaters are async and batched.
- Lift state up to the closest common ancestor when several components share it.
- Derived values belong in regular variables, not state; avoid duplicating the same data in multiple places.

Example

```
function Counter() {
  const [count, setCount] = useState(0);
  return (
    <button onClick={() => setCount(c => c + 1)}>
      Count: {count}
    </button>
  );
}
```

Summary

Components, props, and state form React’s mental model. Props pass data down, state owns mutable data, and re-renders keep the screen in sync with your data model.

Practice Checklist

- Split a page into components and pass data between them via props.
- Convert a mutable value to state and observe how the UI updates.
- Lift shared state to a common parent so two sibling components stay in sync.